

COMPLETE PROTEIN-LIGAND MD PIPELINE (FROM SCRATCH)

Initial folder

clean.pdb
protein_only.pdb
ligand.gro
ligand.itp
ions.mdp
minim.mdp
nvt.mdp
npt.mdp
md.mdp
topol_Protein_chain_A.itp
posre_Protein_chain_A.itp

STEP 1. Generate protein topology

gmx pdb2gmx -f protein_only.pdb -o protein.gro -water spce -ignh

STEP 2. Insert ligand into protein

gmx insert-molecules -f protein.gro -ci ligand.gro -o complex.gro -nmol 1

STEP 3. Define simulation box

gmx editconf -f complex.gro -o complex_box.gro -c -d 1.0 -bt cubic

1.0 nm is standard. Smaller is risky.

STEP 4. Solvate system

```
gmx solvate -cp complex_box.gro -cs spc216.gro -o complex_solv.gro -p topol.top
```

This updates solvent count automatically.

STEP 5. Edit topology, THIS IS NOT OPTIONAL

Add the ligand.itp definitions in topol.top

```
98840
98841 [ molecules ]
98842 ; Compound      #mols
98843 Protein_chain_A   1
98844 UNK               1
98845 SOL             237398
98846 CL               3
98847
```

Add "UNK" 1" in topol.top near the end of the file
"UNK" is the name of the ligand.
check the same in ligand.itp
In this case ligand.itp had:

```
37 | opls_818 C818 12.0110 6
38 [ moleculetype ]
39 ; Name          nrexcl
40 UNK             3
41 [ atoms ]
42 ; nr           type resnr residue
```

```
18 ;
19
20 ; Include forcefield parameters
21 #include "oplsaa.ff/forcefield.itp"
22 #include "ligand.itp"
23
```

Add "ligand.itp" file below the existing "#include" definitions.

5.1 Include ligand.itp

Open `topol.top`

Add **below** forcefield include:

```
#include "ligand.itp"
```

5.2 Add ligand to [molecules] section

At the bottom:

```
[ molecules ]
Protein_chain_A  1
UNK              1
SOL              XXXX
```

- UNK must exactly match ligand name in `ligand.itp`
 - If this is wrong, simulation is invalid
-

STEP 6. Energy minimization

Preprocess

```
>gmx grompp -f minim.mdp -c complex_solv.gro -p topol.top -o em.tpr -maxwarn 1
```

Run

```
>gmx mdrun -v -deffnm em
```

Check potential energy

```
>gmx energy -f em.edr -o em_potential.xvg
# select: Potential
```

```
>xmgrace em_potential.xvg
```

Energy must smoothly decrease.

STEP 7. Create index file (CRITICAL FOR COMPLEX)

```
>gmx make_ndx -f em.gro -o index.ndx
```

Inside prompt:

```
>1 | 13  
>q
```

This creates the Protein + **Ligand group**.

STEP 8. Visualize EM result

```
>gmx trjconv -s em.tpr -f em.gro -n index.ndx -o em_centered.gro -center -pbc mol
```

Selections:

- Center: Protein_UNK
- Output: System

```
>vmd em_centered.gro
```

Visually confirm ligand is bound.

STEP 9. NVT equilibration

Preprocess

```
>gmx grompp -f nvt.mdp -c em.gro -r em.gro -p topol.top -o nvt.tpr -maxwarn 1
```

Run

```
>gmx mdrun -v -deffnm nvt
```

Temperature check

```
>gmx energy -f nvt.edr -o nvt_temperature.xvg  
# select: Temperature
```

```
>xmgrace nvt_temperature.svg
```

Must stabilize around target temperature.

STEP 10. NVT visualization

```
>gmx trjconv -s nvt.tpr -f nvt.trr -n index.ndx -o nvt_center.xtc -center -pbc mol -ur compact
```

```
>vmd nvt.gro nvt_center.xtc
```

Check ligand stability again.

STEP 11. NPT equilibration

Preprocess

```
>gmx grompp -f npt.mdp -c nvt.gro -r nvt.gro -t nvt.cpt -p topol.top -o npt.tpr -maxwarn 1
```

Run

```
>gmx mdrun -v -deffnm npt
```

Pressure

```
>gmx energy -f npt.edr -o npt_pressure.svg  
# select: Pressure
```

Density

```
>gmx energy -f npt.edr -o npt_density.svg  
# select: Density
```

Density should settle near 1000 kg/m³.

STEP 12. Production MD

Preprocess

```
>gmx grompp -f md.mdp -c npt.gro -t npt.cpt -p topol.top -o md.tpr -maxwarn 1
```

Run

```
>gmx mdrun -v -deffnm md
```

STEP 13. Clean trajectory (MANDATORY)

```
>printf "4\n0\n" | gmx trjconv -s md.tpr -f md.xtc -o md_noPBC.xtc -center -pbc mol -ur compact
```

```
>printf "4\n0\n" | gmx trjconv -s md.tpr -f md_noPBC.xtc -o md_fit.xtc -fit rot+trans
```

STEP 14. RMSD analysis (protein backbone)

```
>printf "4\n4\n" | gmx rms -s md.tpr -f md_fit.xtc -n index.ndx -tu ns -o rmsd_backbone.xvg
```

STEP 15. Visualize PRODUCTION MD trajectory

You already created the correct trajectory: `md_fit.xtc`.

Load in VMD (correct way)

```
>vmd md.gro md_fit.xtc
```

If you want center-of-mass smooth motion:

```
>vmd md.gro md_noPBC.xtc
```

STEP 16. Plot RMSD properly (no hand waving)

View RMSD

```
>xmgrace rmsd_backbone.xvg
```

What a GOOD RMSD looks like

- Rises initially (equilibration)
- Plateaus
- No continuous upward drift

If RMSD keeps increasing linearly, the system is unstable.

STEP 17. Overlay RMSD for presentation

If you want RMSD in **nanoseconds**:

```
>gmx rms -s md.tpr -f md_fit.xtc -n index.ndx -o rmsd_ns.svg -tu ns
```

Open both:

```
>xmgrace rmsd_backbone.svg rmsd_ns.svg
```

Pick the cleaner one for paper.

STEP 18. Ligand stability visualization (not optional)

Ligand RMSD relative to protein

```
>gmx rms -s md.tpr -f md_fit.xtc -n index.ndx -o rmsd_ligand.svg
```

Selections:

- Reference: Protein
- RMSD group: UNK

This answers the real question: **does ligand stay put.**

Plot:

```
>xmgrace rmsd_ligand.svg
```

STEP 19. Distance based sanity check

Distance between ligand and protein COM:

```
>gmx distance -s md.tpr -f md_fit.xtc \  
-select 'com of group Protein plus com of group UNK' \  
-oall lig_prot_dist.svg
```

Plot:

```
>xmgrace lig_prot_dist.svg
```

Flat = stable binding

Spike = ligand drift

Reviewers love this.

STEP 20. Make a clean MD movie (for sanity + presentation)

Create movie friendly trajectory

```
gmx trjconv -s md.tpr -f md_fit.xtc -o md_movie.xtc -skip 10  
>
```

Load:

```
>vmd md.gro md_movie.xtc
```

Then:

- Representation: Cartoon + Licorice for ligand
 - Color protein by chain
 - Color ligand by element
-

STEP 21. Optional but strong plots

RMSF (residue flexibility)

```
>gmx rmsf -s md.tpr -f md_fit.xtc -o rmsf_ca.svg -res
```

Radius of gyration

```
>gmx gyrate -s md.tpr -f md_fit.xtc -o gyration.svg
```
